

Finance Quest — Retrospectiva Técnica da Etapa 1

Etapa: SQLite + SQL + primeira integração com Python

****Status:**** Concluída

****Próxima etapa:**** Python + arquitetura da aplicação

1. Objetivo desta retrospectiva

Registrar o que foi efetivamente construído na primeira etapa prática do Finance Quest, avaliar os conhecimentos demonstrados no código e identificar os pontos que devem ser consolidados antes da transição para a construção da aplicação.

Esta retrospectiva deve ser mantida como documentação de desenvolvimento e aprendizagem do projeto.

2. Resultado alcançado

A Etapa 1 transformou o domínio financeiro do Finance Quest em um banco de dados relacional funcional, utilizando SQLite e scripts Python.

A estrutura atual contempla, entre outras, as entidades:

- Instituição
- Conta
- Cartão
- Categoria
- Pessoa
- Meio de Pagamento
- Movimentação
- Compra
- Item da Compra
- Parcela
- Orçamento
- Meta
- Participação/Rateio

O projeto já possui scripts separados para criação, inserção, consulta e reset do banco.

3. Modelo conceitual atualmente implementado

```
INSTITUIÇÃO
/ \
/ \
CONTA  CARTÃO
 \ /
 \ /
MEIO_PAGAMENTO
|
|
MOVIMENTAÇÃO
```

e:

```
instituicao.id
↑
■
cartao.id_instituicao
```

4.4 INSERT — ■

O projeto já utiliza scripts Python para inserir dados nas entidades.

4.5 SELECT — ■

Já existem consultas para recuperar dados do banco.

4.6 JOIN — ■

Foi implementada consulta envolvendo cartão e instituição:

```
SELECT cartao.nome, cartao.limite_total, instituicao.nome
FROM cartao
JOIN instituicao
ON cartao.id_instituicao = instituicao.id;
```

Isso demonstra a compreensão inicial de relacionamentos relacionais e recombinação de dados.

4.7 Python consumindo resultados SQL — ■

Os resultados retornados pelo SQLite já são percorridos no Python, utilizando índices e f-strings para apresentar informações.

4.8 Soft delete / desativação — ■

A utilização de campos como:

```
ativo INTEGER DEFAULT 1
```

está alinhada à regra de negócio de desativar entidades em vez de apagá-las.

5. Principais pontos fortes

5.1 Tradução do domínio real para entidades

O projeto não foi construído como um exercício artificial. Ele representa problemas financeiros reais:

- contas;
- cartões;
- compras;
- parcelas;
- orçamento;
- metas;
- pessoas;
- rateios.

5.2 Separação Compra → Item → Parcela

Essa modelagem é particularmente importante para o futuro painel detalhado dos cartões, permitindo compreender a composição de uma fatura e acompanhar parcelas.

5.3 Pensamento de integridade

O uso de PK, FK, UNIQUE, CHECK, DEFAULT e CASCADE mostra que o projeto já começou a tratar o banco como uma estrutura com regras, e não simplesmente como uma coleção de tabelas.

5.4 Evolução incremental

A construção foi feita em etapas, permitindo aprender progressivamente:

```
Instituição/Conta/Cartão
↓
Categorias
↓
Pessoas
↓
Meios de pagamento/Movimentações
↓
Compra/Item/Parcela
↓
Orçamento/Meta/Participação
```

6. Pontos de modelagem a consolidar

Estes pontos não devem ser tratados simplesmente como “erros”. São decisões de modelagem que precisam ser formalizadas antes da versão definitiva.

6.1 Tipo de movimentação

O modelo atual utiliza a categoria para representar a natureza da movimentação.

Precisamos decidir se isso será suficiente ou se a movimentação deverá possuir explicitamente:

```
Entrada
Saída
```

Status: ■ decisão pendente.

6.2 Meio de pagamento

A entidade possui referências para conta e cartão.

É necessário reforçar a integridade para impedir combinações incoerentes, como um meio do tipo cartão apontando para conta e sem cartão.

Status: ■ bom modelo inicial; constraints precisam ser reforçadas.

6.3 Parcelas

A estrutura atual possui compra, número da parcela, vencimento, valor e situação de pagamento.

Precisamos formalizar regras como:

- uma compra não possuir duas parcelas com o mesmo número;
- determinar conscientemente se o total de parcelas será armazenado ou derivado.

Status: ■ estrutura boa; regras precisam ser refinadas.

6.4 Participação / rateio

A estrutura permite representar cotas de pessoas em uma compra.

Precisamos formalizar:

- quem efetivamente pagou;
- quanto cabia a cada pessoa;
- quem deve ressarcir quem;
- como calcular o saldo líquido entre participantes.

Status: ■ modelo promissor; regra de negócio precisa ser formalizada.

6.5 Orçamento

O orçamento atualmente relaciona categoria, mês/ano e valor limite.

Deve existir uma regra que impeça dois orçamentos para a mesma categoria no mesmo período.

Possível regra:

```
UNIQUE(categoria, mes_ano)
```

Status: ■ precisa de constraint composta.

6.6 Metas

A entidade Meta atualmente representa uma versão inicial com nome, valor-alvo e data-limite.

O PRD prevê evolução para incluir:

- prioridade;
- ativo/inativo;
- aporte mensal planejado;
- conta onde os recursos estão alocados;
- valor acumulado;
- movimentação de recursos entre metas;
- rentabilidade.

Status: ■ MVP inicial; ainda incompleta em relação ao PRD.

6.7 Cartões

O banco já armazena informações estruturais como limite, fechamento e vencimento.

Indicadores como:

- limite utilizado;
- limite disponível;
- fatura;
- percentual comprometido;
- Mana do cartão;

devem preferencialmente ser calculados a partir dos lançamentos, em vez de armazenados como dados redundantes.

Status: ■ estrutura inicial adequada; cálculos virão posteriormente.

7. Integridade referencial

Antes da aplicação avançar, é importante consolidar o uso de:

```
PRAGMA foreign_keys = ON;
```

no SQLite.

Isso garante que as foreign keys sejam efetivamente verificadas durante a execução da aplicação.

Esse ponto será utilizado como oportunidade de aprendizagem prática sobre integridade referencial.

8. Scripts de setup

A divisão atual em vários scripts `setup\_\*.py` foi adequada para a fase de aprendizagem.

Entretanto, à medida que o projeto evoluir para uma aplicação, será necessário separar claramente:

```
schema / estrutura
+
migrations
+
seed / dados iniciais
```

O objetivo não é simplesmente “deixar o código bonito”, mas tornar a aplicação reproduzível e segura para evolução do banco.

9. Reset do banco

O `reset.py` é útil como ferramenta de laboratório durante o desenvolvimento.

No produto final, operações destrutivas não devem estar disponíveis de forma indiscriminada.

Futuramente, o projeto poderá evoluir para mecanismos mais controlados de:

- reset de ambiente de desenvolvimento;
- seed de dados;
- migrations;
- backup.

10. Diagnóstico de aprendizagem

Competência	Situação
Python básico	■
sqlite3	■
Conexão Python/SQLite	■
CREATE TABLE	■
PK	■
FK	■
NOT NULL / UNIQUE	■
CHECK	■
DEFAULT	■
INSERT	■
SELECT	■
fetchall	■
JOIN básico	■
Loops Python	■
F-strings	■
Modelagem relacional	■/■
Cardinalidade	■
Normalização	■
Constraints avançadas	■

Competência	Situação
Consultas analíticas	■/■
Funções Python	■
Tratamento de erros	■
Arquitetura de aplicação	■
Pandas	■
Data Quality	■
Data Matching	■
Dashboard	■
Automação	■

Este diagnóstico não é uma prova ou julgamento. É um mapa para orientar a aprendizagem.

11. O principal aprendizado da Etapa 1

O maior avanço não foi apenas aprender sintaxe SQL.

Foi aprender a transformar problemas reais em:

```
ENTIDADES
+
ATRIBUTOS
+
RELACIONAMENTOS
+
REGRAS
```

O Finance Quest começou como uma necessidade de controle financeiro e já está sendo transformado em um sistema de dados.

Esse raciocínio será fundamental para a futura atuação em análise de dados.

12. O que ainda não será construído nesta etapa

Para evitar atropelar o aprendizado, ficam fora da Etapa 1:

- Dashboard;
- Streamlit;
- Pandas;
- gráficos;
- RPG;
- integração com APIs bancárias;
- automação bancária;
- cloud;
- aplicativo mobile;
- IA e recomendações.

Esses componentes serão construídos sobre a base atual.

13. Mini-fase 1.5 — Consolidação recomendada

Antes da arquitetura da aplicação, recomenda-se uma pequena fase de consolidação com desafios usando o próprio banco.

SQL

1. Listar cartões.
2. Mostrar instituição de cada cartão.
3. Total gasto por categoria.
4. Total gasto por cartão.
5. Total das parcelas futuras.
6. Categoria mais próxima do orçamento.
7. Parcelas que terminam primeiro.
8. Valor potencialmente liberado quando parcelas terminarem.

Python

Transformar consultas em funções:

```
def listar_cartoes():  
    ...  
  
def listar_compras():  
    ...  
  
def calcular_fatura():  
    ...  
    ...
```

Integridade

Estudar e praticar:

- PRAGMA foreign_keys;
- CHECK;
- UNIQUE composto;
- transações;
- rollback.

Modelagem

Revisar conscientemente:

- Movimentação;
- Meio de pagamento;
- Parcelas;
- Rateio;
- Orçamento;
- Metas;
- Cartões.

14. Transição para a Etapa 2

A próxima etapa muda o foco de scripts isolados para uma aplicação organizada.

Modelo conceitual futuro:

```
Interface
↓
Regras de negócio
↓
Acesso a dados
↓
SQLite
```

Estrutura inicial possível:

```
Finance-Quest/
■
■■■■ app.py
■
■■■■ database/
■ ■■■ connection.py
■ ■■■ schema.sql
■
■■■■ models/
■
■■■■ repositories/
■
■■■■ services/
■
■■■■ queries/
■
■■■■ dashboard/
■
■■■■ tests/
```

A arquitetura definitiva será construída gradualmente e explicada durante o desenvolvimento.

15. Veredito da Etapa 1

Status: APROVADA / CONCLUÍDA

Nível desbloqueado

■ Aprendiz de Dados — SQL & SQLite

O projeto já possui uma base suficiente para começar a aprender análise de dados e desenvolvimento de aplicações usando problemas reais do Finance Quest.

A próxima fase deverá preservar a filosofia:

****Não construir o Finance Quest apesar de estar aprendendo. Construir o Finance Quest justamente para aprender.****

16. Registro de evolução

Etapa 1: SQLite + SQL + Python básico

****Resultado:**** Banco relacional funcional + primeiros scripts de consulta e inserção

****Próximo marco:**** Consolidação SQL/Python

****Depois:**** Arquitetura da aplicação

****Objetivo futuro:**** Finance Quest como assistente pessoal financeiro, analytics + automação + gamificação RPG 32-bit.